

**Programa de Ingeniería de Software**  
**Programación con Tecnologías Web**  
**V Semestre**  
**Parcial 1**

**Docente: Santiago Jaramillo López**

## **Descripción**

---

Desarrollar una página web para promocionar y vender un producto de su elección. El producto puede ser cualquier cosa: ropa, tecnología, comida, servicios, etc. Lo que se evalúa no es el producto sino la calidad técnica y de usabilidad de lo que se construya.

Puede trabajar de forma individual o en pareja.

**La entrega es el link del repositorio de GitHub enviado al correo del docente antes del viernes 15 de mayo a las 11:59 PM. No hay sustentación: lo que se entrega es lo que se califica.**

## **Requerimientos mínimos obligatorios**

---

### **A. Estructura y semántica HTML**

- La página debe tener mínimo 3 secciones o vistas con interacción real del usuario (por ejemplo: landing con presentación del producto, catálogo o detalle, formulario de compra o contacto).
- Uso correcto de etiquetas semánticas de HTML5: header, nav, main, section, article, footer según corresponda. No usar divs genéricos para todo.
- Jerarquía de encabezados coherente: un solo h1, h2 para secciones principales, h3 para subsecciones.
- Los formularios deben tener label visible asociado a cada campo (no reemplazar el label con placeholder). Usar el atributo type correcto en cada input: email, number, tel, date según el dato que se pide.

### **B. Modelo de caja y diseño responsivo**

- El layout no debe tener elementos desbordados ni anchos rotos.
- La página debe ser completamente responsiva: debe verse y funcionar correctamente tanto en pantalla de escritorio como en pantalla de móvil. Usar media queries, Flexbox o Grid según sea necesario.
- El espaciado entre secciones debe ser coherente y usar márgenes/paddings con criterio, no a ojo.

### **C. Usabilidad**

- Los botones deben tener texto específico que describa exactamente la acción que ejecutan. No usar 'OK', 'Enviar', 'Aceptar' ni similares.
- Los mensajes de error deben decir qué falló y cómo corregirlo. No mostrar códigos técnicos ni mensajes genéricos.
- Las operaciones que toman tiempo deben mostrar un estado de carga visible (el botón se deshabilita y cambia su texto mientras se procesa).
- Las acciones irreversibles (eliminar, cancelar un pedido) deben pedir confirmación antes de ejecutarse.
- Los estados vacíos deben tener un mensaje explicativo, no un espacio en blanco.
- La jerarquía visual de la página debe ser clara: el usuario debe saber inmediatamente cual es la acción principal de cada sección.

## D. API simulada

El proyecto debe incluir un archivo `mockApi.js` que simule las respuestas de la API del sistema. El front debe consumir este archivo como consumiría una API real.

- Mínimo 2 endpoints simulados: uno para consultar datos (equivalente a un GET) y uno para registrar una acción del usuario como una compra, suscripción o contacto (equivalente a un POST).
- Cada función en `mockApi.js` debe simular la latencia de red con un delay antes de resolver (mínimo 500ms) para que los estados de carga sean visibles y evaluables.
- El `mockApi.js` debe simular tanto la respuesta exitosa como el caso de error, y el front debe manejar ambos casos correctamente.
- Los endpoints deben estar documentados en el README: nombre de la función, que equivalente HTTP representa (método y URL siguiendo las convenciones vistas en clase), qué recibe como parámetro y qué devuelve en éxito y en error.

## E. Repositorio y código

- El código debe estar en un repositorio público de GitHub.
- Libertad de elección del framework o tecnologías para trabajar.
- El README debe incluir: descripción breve del proyecto, como correrlo localmente, y la documentación de los endpoints simulados en `mockApi.js`.

## Criterios de evaluación

| Criterio                                 | Qué se evalúa                                                                                                                                                               | Peso        |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Semántica HTML                           | Uso correcto de etiquetas estructurales, jerarquía de encabezados, formularios con labels y types apropiados.                                                               | 15%         |
| Modelo de caja y responsividad           | Layout sin elementos rotos, diseño responsivo funcional en escritorio y móvil.                                                                                              | 15%         |
| Usabilidad                               | Texto de botones específico, labels correctos, mensajes de error informativos, estados de carga, confirmación de acciones irreversibles, estados vacíos con mensaje.        | 25%         |
| API simulada ( <code>mockApi.js</code> ) | Mínimo 2 funciones simulando GET y POST, con delay, manejo de éxito y error en el front, documentadas en el README siguiendo las convenciones de endpoints vistas en clase. | 25%         |
| Calidad del código y repositorio         | Estructura de archivos organizada, código limpio, README completo, repositorio público en GitHub.                                                                           | 20%         |
| <b>TOTAL</b>                             |                                                                                                                                                                             | <b>100%</b> |

## Notas importantes

1. No se aceptan entregas por fuera del plazo. El link debe llegar al correo del docente antes del viernes 15 de mayo a las 11:59 PM.
2. El repositorio debe ser público. Si el docente no puede acceder al repositorio, la entrega se califica con cero.
3. Si se desarrolla en parejas el parcial, ambos integrantes del equipo deben tener aportes en el repositorio.